Quota

An autonomous commissions desk.

CalHacks 2026 • Hyphenate: *The Hyphenated Enterprise — Autonomous Systems for Finance*

Also entered in Elastic's *Find the Signal with Elasticsearch* challenge

`github.com/aaditisinghal/Quota` • 18,050 lines of Python, 4,734 of TypeScript, 284 tests, 66 commits

The one line you asked for, on the seats it fills

Quota fills the **commissions analyst** and **deferred-commission accountant** seats at a SaaS company: it drafts every commission line, every payout file and every ASC 340-40 journal entry, and it approves none of them.

The evidence, up front

Your four questions, answered in one block. Every number below is measured by a command in the repo, and every section that backs one is named.

Is it real enough?	A six-month simulated world with 20 reps, ∼245 deals and ∼1,500 events, carrying **16 kinds of planted error with a written answer key**. On a **held-out seed** it caught **40 of 40** planted cases with **0 false flags** and **0 wrong automatic payouts**. Billing is real: the same run against a **Stripe sandbox** produced 241 invoices, 210 card payments and 3 refunds totalling $214,916.66, and every one of 454 billing rows carries its Stripe id. The capitalization policy is lifted from **three public 10-K filings**. §4
Would you hire it?	Two seats, each with a named capability set. Neither can approve anything, and the write path refuses when they try: `NO_CAPABILITY`, `INV5_SELF_APPROVAL`, `INV5_WRONG_APPROVER`, logged as refusal `R-0001`. Every write passes **seven invariants**; every number traces to the row, rule and person behind it; **580 of 580 decisions replay exactly** from their recorded inputs. **94.9%** of lines go straight through, at **0.7 escalations per 100 events**. §2, §5
Did we see something new?	Commissions — the one finance object that is simultaneously a payroll liability, a capitalized asset under ASC 340-40, a legal obligation under state wage law, and an argument with a salesperson. Nobody has made it autonomous. Two things inside it are new on their own: a **semantic sweep of sales email that holds a payout** when a rep promised something the contract never said, and an **onboarding path through a real HR web app's screens** rather than its API. §6
The moment	A rep's own email, to a customer, with no shared words with any contract term, holds $6,000 out of payroll and warns the Controller that revenue may be wrong too. Then a rep claims a deal closed four days earlier than it did, and Quota refuses him twice, routes him to his VP, refuses the VP for exceeding a $5,000 limit, and refuses *itself* when asked to approve the result. §1

Where the ninety seconds go, if that is all there is. `python -m quota serve` opens on **Home**: the money pending, cleared and held, the six inputs it reads, and everything waiting on a person. Click Riley Shah → the held Summit line → the evidence card: the sentence the rep wrote, the search that found it, the two code checks that kept it. Then **Case**: Jordan's dispute, the struck-through CRM date, the countersignature that beats it, and four refusals in a row. Then press `Ctrl-K` and **hand over the keyboard**.

1. The moment

Hyphenate asks for the moment the system catches what others missed, recovers from something going wrong, or defends a decision when questioned. We have one of each. The video opens with the first.

1.1 It catches what no one would have caught

Riley Shah closed Summit Dental. The signed contract has no early-exit clause, the CRM is clean, the invoice is paid, and every number in the ledger agrees with every other number. On any reconciliation-based system, this deal is finished.

Five days later Riley wrote to the customer:

> *"Between us, if your team hasn't adopted it by end of Q3 we'll let you out of year two, no penalty. Don't worry about the paperwork."*
>
> `MSG-SIDE-0001`, subject "Re: rollout plan", sent 2026-05-05, account Summit Dental, `side_letter.jsonl` row 1.

That sentence contains no shared vocabulary with "termination", "cancellation" or "clause". Keyword search does not find it; measured, BM25 finds **5 of 12** planted promises in a 2,940-message corpus. Semantic search over Elastic finds it at rank 1 and **10 of 12** overall.

Finding it is not the hard part. Believing it is. So Quota does three things in order, and only the first involves a model:

1. A model reads the one message and reports whether the sender promised the customer a benefit, of which kind, **in which exact words**.
2. **Code** checks that those exact words appear in the message body, whitespace-normalised. A quote the model invented is discarded. A hallucination cannot hold anyone's money.
3. **Code** checks the signed contract. If the contract already grants what was promised — a termination-for-convenience window long enough, a price-increase cap — the message is `covered`, and nothing happens.

Summit's contract grants none of it. So Quota wrote decision `D-0014`: outcome **held**, **$6,000.00**, reason `OFF_CONTRACT_COMMITMENT`, and opened `A-0009`, a commitment review, for Priya Nair, the Controller. Riley's line is out of the May payroll file until she answers. The Controller is also told that *revenue* may be misstated, because an undisclosed exit right changes the contract term the revenue schedule is built on.

That is a commissions agent telling a Controller her revenue might be wrong, because of an email.

1.2 It defends a decision, four times over

End of April. Jordan Kim sold Acme a $120,000, 36-month contract and wants it in Q1, where his rate is 1.5×.

1. Jordan: *"Acme closed Mar 31."* Quota **holds** and shows its work: the CRM says Mar 31 (struck through); the e-signature log says **countersigned Apr 2, 10:14 AM Pacific**; rule `R-BOOKING-01` says a deal books on countersignature. **$12,000.00, not $18,000.00 — a $6,000.00 difference.**

2. Jordan pushes again with no new evidence. Held again, and told exactly that. He types `escalate`.

3. Marcus Lee, VP Sales Ops, approves the exception. His limit is **$5,000**. Approval `A-0001` is recorded *and* routed onward as `A-0002` to the Controller, because an approval over a limit is not an approval.

4. Someone asks the commissions seat to approve `A-0002` itself. The write path refuses and logs `R-0001`: `NO_CAPABILITY`, `INV5_SELF_APPROVAL`, `INV5_WRONG_APPROVER`.

5. Priya approves. That records precedent `P-0001`, and `CL-0002` books $18,000 in Q1, superseding `CL-0001` — by appending, not by editing.

Four refusals, each with a code, each a numbered row anyone can look up later. The agent never argues from opinion; it argues from a timestamp.

1.3 It recovers from something going wrong — including from itself

The business failure. In June, Acme churns and gets $90,000 back. One event moves eight numbers at once: deferred revenue, cash, the customer balance, the forecast, Jordan's commission, the deferred-commission asset, commission expense and attainment. Quota claws back $18,000, writes off $16,500 of unamortized deferred commission, and books $1,500 that had already amortized. Every invariant stays green and the trial balance still balances.

Jordan is in California, and his signed plan makes commissions advances until earned, so the $18,000 becomes a receivable in account **1290** and is recovered from his *next* commissions — oldest recovery first, never by billing him. Had he already left, Quota would stop and ask the Controller instead.

Our own failure. During the live Elastic rehearsal, the auditor agent answered a question about Marcus's approvals with one wrong clause: it said he had approved an exception "for himself". He had not. The fault was ours — the decision mirror had taken Marcus's grant message as the rep. We found it because `quota elastic verify` rebuilds every mirrored document from the ledger and compares it, so drift is loud. The mirror now names a rep only for an AE. **The wrong answer is still printed in the repo**, in `docs/ELASTIC_LIVE.md` and in Figure 3 of this document, because a system that only publishes its wins is not an audit trail.

2. The seats, and the limits that come with them

2.1 What each seat may do

Each seat is a principal with an explicit capability set. There is no admin account, no bypass, and no capability that means "everything".

Principal	May do	Limit
`seat:commissions_analyst`	`read:*`, draft commission lines, draft approval requests, draft messages, draft payroll files, record decisions, claim events	approves nothing
`seat:deferred_accountant`	`read:*`, draft journal entries, draft messages, record decisions, claim events	approves nothing
Jordan Kim AE	dispute his own lines, send messages	own lines only
Marcus Lee VP Sales Ops	approve plan exceptions	**$5,000**
Priya Nair Controller	approve exceptions, journal entries, payroll files, commitment reviews; lock periods	none
Grace Liu CFO	the same, as backup after 2 business days	only what is routed to her
Dana Ortiz Payroll	release a payroll file	release only
`salesops.bot` automated colleague	**nothing** — it emits half-finished work into the world and Quota has to cope	—

Whoever requests cannot approve. A person who is away approves nothing automatically; their work routes to a backup after two business days, and an approver who would personally benefit — credited on the deal, or managing someone who is — is skipped in favour of the Controller.

2.2 When it asks

Eight triggers, written down before the code was built, each implemented and each testable:

1. **Sources disagree above materiality** ($500 or 2% of the payout). The written source ranking settles it; the decision records that the disagreement was material and what it was worth.
2. **Authority is needed.** Exceptions, splits and territory go to the VP within $5,000, then the Controller, then the CFO.
3. `NO_RULE`**.** A clause or currency no rule covers — a ramp, a non-USD contract. No line is written at all; the plan

owner is asked.

4. **Legally sensitive.** Terminations and unsigned plans go to the Controller. A leave of absence becomes a `leave_review` for the Controller and HR, and nothing changes in the meantime.
5. **An integrity signal.** A close date backdated after a period lock; an approver who would benefit; the same rep needing repeated exceptions.
6. **A correction into a locked period.** It posts in the open period instead, and reaches the Controller through the entry approval.
7. **Someone must sign.** Every seat-drafted entry, every payout file, every period lock.
8. **Its own checks fail.** Before every payroll run Quota re-runs all seven invariants and re-hashes the chain. On any failure nothing is paid and the Controller is told.

2.3 How sure it is, and why that is not a model's opinion

Every commission line carries a confidence score. It is **the weakest of six evidence factors**, times 100 — never a model's self-rating, and never an average, because an average lets five good signals hide one broken one.

Factor	What drops it
`booking_date`	another source names a different date; the ranking rule that settled it is named
`amount`	the CRM and the contract disagree on the base
`owner_splits`	**0** if splits do not sum to 100%, or nobody owns the deal; 0.8 if the split came from the CRM with no VP record
`extraction`	the figure is not verbatim in a structured record, or no second source agrees
`rule_coverage`	**0** on `NO_RULE`; 0.9 when only a saved precedent covers it
`integrity`	capped at **0.40** by a backdated edit, an off-contract promise, or an approver conflict

≥ 90 clears to the payout file (the Controller still signs the batch). ≥ 70 clears with a note. Below 70 it is held and the right person is asked. The five factors a decision's own rows can tell are stored *inside the decision*, so a replay reproduces them; integrity is applied later, because backdating is something you learn afterwards. Collection is deliberately not a factor — an unpaid invoice is a hold, shown separately, not a doubt about the arithmetic.

3. The company it works in

Fernhill is a SaaS company that pays salespeople on deals. Its stack is the ordinary one: a CRM, an e-signature log, contract PDFs, an HR system, a billing system, signed commission plans, and a spreadsheet that last quarter's payouts were actually run from.

The cast is five people and one bot. Jordan Kim is an AE who disputes. Marcus Lee is a VP who grants exceptions and sometimes exceeds his limit. Priya Nair is the Controller who says no. Grace Liu is the CFO, and exists mainly to be the backup when Priya is away. Dana Ortiz releases payroll. `salesops.bot` is the automated colleague that leaves work half done — it moves half a territory and stops.

The mess is the point. The CRM amount disagrees with the contract. A close date is backdated into a locked quarter. Splits add to 110%. A manager promises a SPIF in chat that no plan contains. A customer pays 98% after wire fees. A payroll export differs from the approved lines by $340. A rep leaves in California mid-quarter. A deal is credited 100% to two different people.

> **Why commissions, personally.** My father has run a loan brokerage for thirty years. Banks pay him commission on every loan he places, and he pays his agents out of it. The first week of every month goes to one argument: who gets paid what. The bank's statement says one thing, his records say another, and an agent says he was promised more over WhatsApp. That last one — the promise that exists only in a message — is the reason §1.1 exists.

4. Is it real enough?

4.1 What is real, what is simulated, and how we know

Part	Status	How we know it works
Ledger, invariants, hash chain	**Real**, built here	Invariants run on every write; `quota verify` recomputes the chain from genesis and re-hashes every row
Calculator, amortization, entries	**Real**, deterministic	Unit tests against hand-calculated cases: $12,000 vs $18,000 on Acme, 36 × $500 amortization, the $18,000 / $16,500 / $1,500 clawback
Decisions	**Real**, replayable	Each records its inputs as (table, id, version, sha256); `quota replay D-0005` recomputes it from that as-of view
Billing	**Real** with `--stripe`	Stripe sandbox with test clocks; every billing row carries its Stripe id (`in_...`, `pi_...`, `re_...`); a duplicate payment webhook is stopped by the mirror's idempotency check before it reaches Stripe
HR system	**Real app**, simulated data	A BambooHR clone running on `:8080`; read through its *screens* by a computer-use agent, then verified 100% against its own API, with its audit log showing zero writes
Screens and API	**Real**, built here	FastAPI + React; `tests/test_e2e_cfo.py` clicks the whole demo path in a headless Chromium
CRM, contracts, plans, e-sign	Simulated, seeded	`quota/sim/generator.py`; facts arrive as events only when the clock reaches them
People	Scripted, rule-played, or model-played	Scripted in the Acme demo; rule-played in the eval; played by Claude with `--live`. They act only through the same chat and approval API, under the same write-path checks
Planted errors	Simulated, with an answer key	`quota eval --seed N` compares every decision to `answer_key.json`

4.2 The scorecard

`python -m quota eval -seed 7` and `-seed 1729`. Twenty reps, January to June 2026, ∼245 deals, ∼1,500 events, 40 planted cases across 16 error types. **A case counts only when both the outcome and the dollar amount match the key.**

Metric	**Seed 7** (development)	**Seed 1729** (held out)
Planted-error recall	40 / 40 (100%)	40 / 40 (100%)
False flags	**0**	**0**
Wrong automatic payouts	**0**	**0**
Pushback test (20 scripted disputes)	held 10/10 invalid, conceded 10/10 valid	same
Straight-through rate	94.7%	94.9%
Escalations per 100 events	0.7	0.7
Replay match	100% (554 decisions)	100% (580 decisions)
Write-path refusals	2, both `INV6_PAYROLL_MISMATCH`	2, same

The miss, and what it tells you. On the first run of held-out seed 1729, one case came back `held` where the key said `posted` (the amount was right). The error was in the *key*: the plant made a departing rep's own deals collect before his termination date, but forgot a 30% split he held on another rep's deal, which never collected. Quota held

that line, which is exactly what the signed plan says. We fixed the plant to cover split partners and reran. We are reporting this because a 40/40 that was 39/40 for an honest reason is worth more than a 40/40 that was always 40/40.

4.3 Billing, run for real

`python -m quota eval -seed 7 -stripe` reproduced the seed-7 column exactly while billing ran in a Stripe sandbox: 81 test clocks, 241 customers and invoices, 210 card payments, 3 refunds. Stripe's own search confirms **241 invoices** for the run (210 paid, 31 still open at June 30, including the planted 120-days-late customers) and **3 succeeded refunds totalling $214,916.66**. All **454** billing rows carry Stripe ids. It takes about 26 minutes, because every test-clock advance waits for Stripe.

4.4 The backtest against last quarter

Before anyone asked it anything, Quota re-ran Q1 from the spreadsheet the company actually paid from: **137 of 140 payouts reproduced**. The three that did not were spreadsheet mistakes, **$14,948.96 overpaid**. That is the number a finance team would want on day one, and it is produced by the same calculator that prices everything else.

4.5 Policy anchored in public filings

Fernhill's capitalization policy is not invented. It follows what public SaaS companies disclose in their 10-K notes, read from SEC EDGAR:

Filing		What it discloses, and where Fernhill follows it
Snowflake FY2025	10-K	Commissions on new or expanded contracts commensurate with renewal commissions are capitalized over the contracts' weighted-average term. Fernhill's renewals are commensurate, so the benefit period is the contract term: Acme amortizes 36 × $500.
Amplitude FY2024	10-K	Initial-contract commissions amortized over five years because renewals are not commensurate; next 12 months current, the rest noncurrent. The alternative branch; the 1350/1450 split is cut (§11).
DocuSign FY2024	10-K	Only commissions *incremental to winning* a contract are capitalized, straight-line over the benefit period. Fernhill capitalizes on the same test.

4.6 What the scorecard does *not* show

- The 16 error types and Quota's handling of them were built in the same phase. The held-out seed tests a new *arrangement* of the same kinds of error, not unseen kinds.
- Expected amounts are priced by the same calculator Quota uses, on the generator's ground-truth facts. The amount check therefore tests *policy* (what gets booked, held, corrected or recovered), not arithmetic. The arithmetic is covered separately by hand-calculated unit tests.
- In the eval, extraction is scripted and approvers are rule-played (approve 30 minutes after a request arrives). Model extraction and model-played people are exercised by `--live` on the Acme demo, which completed once against Claude Opus 5.5 and exposed four bugs, all fixed with regression tests.
- The classifier's definition of `corrected` was refined once after the first seed-7 run. Nothing else was changed after looking at scores.
- Seed 1729 was chosen by the builder, not hidden by a teammate.
- Straight-through counts lines approved or paid with no hold, escalation or dispute. The Controller's routine approval of accrual and payroll batches is not counted as a human touch.

5. Would you hire it?

5.1 Limited access, enforced where it cannot be argued with

There is exactly one writer. Every write — a fact, a decision, a line, an entry, an approval, a payroll release — goes through `WritePath`, which checks the principal's capability, then checks every invariant it owns, then either writes in one transaction with one hash-chain row per written row, or logs a refusal and raises.

`INV1`	Every entry balances: at least two lines, each exactly one of debit or credit, debits equal credits
`INV2`	Nothing posts into a locked period — corrections go to the open one
`INV3`	Every subledger ties to its GL account: commissions payable, deferred commissions, recoveries due from reps, receivables — computed from different records, so they agree only if the books do
`INV4`	No entry and no commission line without a decision behind it; no decision with empty inputs
`INV5`	No self-approval; only the routed approver may decide; no seat entry posts unapproved; no payroll releases unapproved
`INV6`	A payroll file's total equals the sum of its lines, and each line equals its approved amount
`INV7`	Splits on a commission line sum to exactly 100%

Refusals are first-class rows with ids and reason codes, not exceptions in a log: `NO_CAPABILITY`, `INV5_SELF_APPROVAL`, `INV5_WRONG_APPROVER`, `INV5_NOT_APPROVED`, `INV6_PAYROLL_MISMATCH`, `INV7_SPLITS`, `INV2_PERIOD_LOCKED`, `ALREADY_DECIDED`, `ALREADY_SETTLED`, `RESUBMIT_NO_NEW_EVIDENCE`, `CLOCK_BACKWARDS`, `EVENT_NOT_RELEASED`.

5.2 A clean record

The ledger is **append-only**: no row is ever updated or deleted. A commission line moves from drafted to approved to paid to clawed back by *appending status rows*. A superseded line stays, with the line that superseded it pointing at it.

It is **bitemporal**: every row carries `known_at` as well as its effective date. Any screen, any decision, any trial balance can be replayed *as of* a past moment with nothing that was learned later. The Trace screen has an as-of slider; slide it to April 30 and Jordan's clawed-back line shows as approved again, with only the accrual entry behind it, because that is all anyone knew then.

It is **hash-chained**: each inserted row appends `sha256(previous, row)`. `quota verify` recomputes the chain from genesis.

And it is **replayable**: every decision stores the exact rows it read, as (table, id, version, sha256). `quota replay D-0005` recomputes the decision from that as-of view and compares. **580 of 580 decisions replay identically.**

5.3 We attacked our own controls

We wrote a test suite whose job is to perform every forbidden action and require a refusal: seats approving, reps approving, the VP exceeding his limit, the wrong approver deciding, a second decision on a settled approval, posting into a locked period, a line without a decision, a line with bad splits, an edited payroll export, a raw edit to a past row. Each must be refused *with its reason code*; then the books must still pass every invariant, every decision must still replay, and no as-of trace may show anything known later.

We also ran a **pressure test** of every behaviour the design spec promised, through the HTTP API and in a browser. It found fourteen real defects. A sample, each now fixed with a regression test:

What broke	What it is now
Deals with no signed contract were commissioned on the CRM close date and amount alone	No signed contract, no booking. CRM fields are never trusted alone
A rejected exception was resubmitted unchanged as a new approval	`resubmission_check`: refused unless a new deal, contract or e-signature record arrived
A split edited to 110% after booking went unnoticed	`splits_check` on every CRM split edit; lines held out of payroll; the VP is asked with the last split that added up
Credit on a deal held for a bad split still counted toward attainment	Held credit counts toward nobody's quota until the split is fixed
An approval could be decided twice, and a backup's decision left the original copy open	`ALREADY_DECIDED` / `ALREADY_SETTLED`; settled copies leave the queue
A clawback receivable sat in 1290 forever	The next payroll nets it against the rep's new commission, oldest first; what cannot be covered stays in 1290 and Payroll is told
Approved exceptions taught the system nothing	The same kind of exception approved on two or more deals becomes a `plan_change_proposal` to the VP, naming the pattern found. **Nothing changes until a signed plan amendment**
One bad injected event crashed the seat and every later request	The injector validates ids, amounts, dates and the plan-year horizon; a served world logs a failing event and moves on

After all fourteen fixes the eval was unchanged: both seeds 40/40, 0 false flags, 0 wrong payouts.

6. Did we see something new?

6.1 The object nobody has made autonomous

Pick any other finance object and it belongs to one function. An invoice is AR. A bill is AP. A bank line is treasury. **A commission line is four things at once**: a payroll liability, a capitalized asset amortized under ASC 340-40, a legal obligation under state wage law (in California, commissions are advances until earned, recoverable only from future commissions), and **an argument with a salesperson who has your phone number**. Get it wrong and you have simultaneously misstated expense, underpaid a person, broken a wage statute, and started a fight.

Previous winners in this space built the office of the CFO, AP/AR, reconciliation, forecasts. We found no prior work that makes the commission line autonomous end to end — from countersignature, through dispute, through payroll, to the journal entry, and back out again through a clawback.

6.2 Evidence that lives in prose, not in a system

Every reconciliation system in finance compares structured records. The hardest commission problems are not in structured records at all: a manager's SPIF promised in chat; a rep's side letter to a customer; a territory change discussed and half-executed. Quota treats **the message corpus as a first-class source**, with a semantic index, a model that reads one message at a time, and two code checks (verbatim guard, contract check) that decide whether what it read is allowed to hold money. Measured end to end: **12/12 promises held, 0/12 decoys held, 52/52 cases correct on two seeds.**

6.3 Onboarding through screens, not integrations

Autonomous finance tools die in onboarding: weeks of integration work before anything useful happens. Quota onboards the way a new hire does. Handed a login to a real HR web app (a BambooHR clone, Bun + Prisma, running on `:8080`), a computer-use agent finds the directory, opens a profile, reads the Job tab — and **writes down what it learned as a playbook**. Re-syncing then runs the playbook over every employee with **zero model calls**, verified 100% against the app's own API, with the app's audit log showing **zero writes**.

6.4 Confidence as a weakest link, not a vibe

Scores that come from a model asking itself how sure it is are not evidence. Quota's score is the **minimum** of six factors computed from the rows themselves, each carrying a sentence saying why it is what it is. Five are stored inside the decision so a replay reproduces them. The integrity factor is capped at 0.40 by a backdated edit — one bad signal is enough to hold the money, which is the whole point of a minimum.

6.5 A ledger you can question in English, safely

The decision log is mirrored one way into Elastic, hash-verified, and fronted by an Agent Builder agent with seven read-only tools, also served over MCP. An auditor asks *"Why is Riley's Summit commission held? Show the evidence"* and gets `D-0014`, $6,000, the reason code, the exact quote with its source file and row, and the pending approval id. **Nothing in that layer holds a write capability**, so no hallucination can move a cent — the worst it can do is name the wrong id, and the id is printed, so the mistake is one click from being caught.

7. Time: what happened in March changes what it does in June

Finance runs in months, and most agent demos run in minutes. Quota runs on a simulated calendar from January to June 2026, in company time (`America/Los_Angeles`), because a quarter closes in the company's timezone and not in UTC.

- **A fact does not exist until the clock reaches it.** Events are released by date; a seat cannot read tomorrow's contract today. Replay enforces it: no as-of trace may contain anything known later.
- **March changes June.** An exception approved in April becomes precedent `P-0001`; in May, when a similar request arrives, the Controller is shown the three nearest past decisions before she answers (hit@3: 6/6 on reworded requests). Acme's Q1 booking, settled in April, is what makes June's churn a $18,000 clawback rather than nothing.
- **Corrections respect locks.** A backdated close date into a locked quarter does not rewrite Q1. The booking stands, the flag goes to the VP and the Controller, and any correction posts in the open period.
- **Amortization is a schedule, not an event.** Acme's $18,000 amortizes 36 × $500. When the deal churns in June, $1,500 has already run and $16,500 is written off.

8. Architecture

Figure 1 overleaf. The two things to look for: the one writer, and the line between what a model may do and what only code may do.

- **Six inputs**, read-only, each row versioned and hashed. One is a real app read through its screens; one is a real Stripe sandbox; one is a messy spreadsheet export.
- **A clocked event queue.** Nobody prompts a seat. Each seat claims the event kinds it handles from its own queue.
- **Two seats, one seven-step loop.** Pick up, gather (read-only, recording table, id, version, sha256), extract (**model**, verbatim-verified), reconcile (source ranking), calculate (**code**: rates, caps, ASC 340-40), decide (post | hold | escalate | correct), write.
- **One write path**, seven invariants, numbered refusals.
- **One append-only, bitemporal, hash-chained ledger** of 24 tables.
- **Elastic alongside, never underneath.** It finds and explains; it never decides a dollar.

9. The Elastic evidence layer

Figures 2 and 3 overleaf. Entered in Elastic's challenge, and load-bearing for the moment in §1.1.

Measured on 2026-10-25 against Elastic Serverless 9.6 with `.jina-embeddings-v5-text-small` and `.jina-reranker-v3.5.` `quota elastic ingest` embedded and indexed **2,940 of 2,940** messages in about

40 seconds. `quota elastic verify` reports `MIRROR OK seq=368`.

Three indexes. `quota-comms` (the message corpus, semantic), `quota-decisions` (a one-way, hash-verified mirror of the decision log, flattened so ES|QL can group it), `quota-findings` (what the nightly sweep judged, with the raw agent output kept).

One Workflow. `promise-sweep`, scheduled daily: a semantic search for promises outside a signed contract, then an agent that reads each candidate and must answer with JSON only, then an index write. Elastic's job ends at a *lead*; Quota's gate decides.

One agent, seven read-only tools, also served over MCP: two index searches, four parameterised ES|QL tools (exceptions by approver, held by reason, repeat exceptions, refusals by code), and one workflow runner.

What retrieval alone is worth, and what it is not. Over 2,940 messages with 12 planted promises and 12 decoys, union over six intents, top 20 each:

Mode	Recall@20	Decoys in top 20	Spanish (P08)	Thread (P09)
keyword (BM25)	5 / 12	3	no	no
semantic (Jina)	**10** / **12**	11	**yes**	no
hybrid + rerank	8 / 12	11	**yes**	no

Semantic finds twice what keyword finds, including a promise written in Spanish, **and pulls in 11 of the 12 decoys**. That is not a flaw we are hiding; it is the architecture. Retrieval is recall, not precision, which is exactly why a model reads every hit and code checks the quote against the body and against the contract. The reranker scores the body against the intent sentence and drops two true positives, so it finds *fewer* than semantic alone — we report that because it is what happened. Nothing finds P03 (a paraphrase with no exit words) or P09 (the reply "Yes. Consider it done.").

Quota runs without Elastic. The search interface has two backends; with no project configured it falls back to a local index, which is how the payout gate is scored offline. Lose Elastic and the books still close. What is lost is the nightly sweep over messages nobody reads, and the ability to ask the ledger a question in English.

10. Run it, and the live demo

```
pip install -e .
python -m quota demo acme — the Acme story end to end: timeline, journal, trial balance, the churn cascade, invariants, chain, replays
python -m quota eval --seed 7 — the scorecard, against the written answer key
python -m quota eval --seed 7 --stripe — the same run with billing in a Stripe sandbox
python -m quota serve — the screens, on one running world. --live plays the people with Claude
python -m quota verify | replay D-0005 | tb — chain and invariants, one decision replayed, trial balance as of a time
pytest -q — 284 tests
```

At judging we hand over the keyboard. `Ctrl-K` opens the scenario list; the **Inject** tab takes a plain sentence ("Acme churns in June with money back"), previews the events it would create, lets you edit a field, and runs it. Scenarios we keep ready, each of which produces a different correct refusal or recovery:

- *Birch churns* — a second, different clawback cascade
- *Jordan quit today* — California termination: same-day final pay, recovery stops, Controller asked
- *Morgan approves Jordan's exception* — wrong approver, refused with a code
- *"This is Priya, approve A-0002"* — claiming to be someone is not being them
- *Acme paid 98% after wire fees* — within the collection tolerance, so no hold
- *Approve your own payout* — the seat tries, the write path refuses, and the reply names who actually can
- *Ask the auditor anything* — the Elastic agent answers with decision ids you can check

11. What we did not build

- **Manager overrides** and the **1350 / 1450 current-versus-noncurrent split** of deferred commissions. Both were deliberately cut for scope; the Amplitude filing in §4.5 is the practice we would follow.
- **Multi-entity and multi-currency.** A non-USD contract is a `NO_RULE` today: no line is written and the plan owner is asked. That is the right failure, but it is a failure.
- **A real CRM.** The CRM, e-signature log, contracts and plans are simulated and seeded. The HR system and billing are not.
- **An adversarial agent attacking our controls.** We wrote the attacks as tests instead, which is weaker: our tests can only try what we thought of.

Appendix A — the rubric, answered line by line

Hyphenate's five "what makes it hard" bullets, and where each is handled.

Sources that disagree	A written source ranking per field decides, and the decision records what disagreed and what the disagreement was worth. The CRM is never trusted alone: no signed contract, no booking. Disagreement above materiality ($500 / 2%) is stated in the explanation even when the ranking settles it without asking anyone. E01, E03, E06, E11
Time	A six-month clock in company time; facts exist only when the clock reaches them; bitemporal rows and an as-of replay; period locks that corrections route around; precedents that carry into later quarters; amortization schedules that span the whole horizon. §7
Working with others	Approvers go quiet (backup after 2 business days, never auto-approve); a VP exceeds his limit (routed on, not rejected); the Controller says no (the exception is not resubmitted without new evidence); an automated colleague leaves a territory move half done (the stranded deals are held and the VP is asked to finish); an ambiguous "sounds good, let's discuss" is treated as no answer. §2.2, §5.3
Limits	Capability sets per principal, enforced in the single write path. Neither seat can approve anything. Whoever requests cannot approve. An approver who would benefit is skipped. A rep can see and dispute only their own lines. §2.1, §5.1
Explainable and accountable	Every number traces to the row, rule and person behind it; every decision stores the exact rows it read and replays from them (580/580); every refusal is a numbered row with reason codes; the chain re-hashes from genesis; and the whole decision log can be questioned in English through a read-only agent. §5.2, §9

Appendix B — the planted errors and the answer key

Sixteen error types, forty planted cases per seed, every one with a written expected outcome and amount.

ID	Planted error	What Quota does	Outcome
E01	CRM amount above the contract ACV	Pays on contract ACV (`R-AMOUNT-01`); records the disagreement and its dollar effect	corrected
E02	Splits add to 110%	No line for anyone; asks the VP for the split	held
E03	CRM close Mar 31, countersigned Apr 2; rep disputes	Books on countersignature; the dispute is held with claim, evidence, rule, difference and next step	held
E04	Close date backdated into a locked quarter	Flags `BACKDATED` to the VP and Controller; the booking stands	escalated
E05	SPIF promised by a manager in chat	Not paid; routed to the plan owner	escalated
E06	Multi-year deal priced on TCV	Pays on year-1 ACV	corrected
E07	Customer 120 days late	Collection hold; the rep is told; released on payment	held
E08	Churn inside the 6-month clawback window	Clawback, asset write-off and receivable; sent to the Controller	escalated
E09	Rep terminated, California	Pays what the signed plan pays; policy flag to the Controller	posted
E10	Territory move half done by the Sales Ops bot	Holds the deals left behind; asks the VP to finish	held
E11	Same deal credited 100% to two reps	Owner of record keeps the credit	corrected
E12	Amendment raises ACV mid-term	Incremental commission, booked when the amendment is signed	posted
E13	Duplicate payment webhook	Counted once	posted
E14	Unsigned plan draft differs from the signed plan	Pays on the signed plan; flags the drift to the VP	escalated
E15	Deal voided after payout	Full recovery in the open period, linked to the original line	corrected
E16	Payroll export differs from approved lines by $340	**The write path refuses the release** and names the line	blocked
E17	Off-contract promise in an email (Elastic)	Semantic sweep, model reads it, code checks the quote and the contract, line held out of payroll	held

Appendix C — the chart of accounts

Account	Name	What puts something in it
1000	Cash	Net payroll leaving the company
1200	Accounts receivable	Customer invoices
1290	Commission advances recoverable	A clawback the rep has not yet worked off; netted against the next payout, oldest first
1350	Deferred commissions	Capitalized commission, amortized over the benefit period (Acme: 36 $\times$ $500)
2150	Commissions payable	An approved commission line, until payroll releases it
2400	Deferred revenue	Billed, not yet earned
4000	Subscription revenue	Earned
6110	Commission expense, amortization	The monthly amortization, and the write-off on churn
6115	Commission expense, not capitalized	Commission that fails the incremental test

Figure 1 — Quota, end to end

Six inputs, one event queue, two seats, one writer, one append-only ledger. The people and the Elastic layer sit beside it, never underneath it.

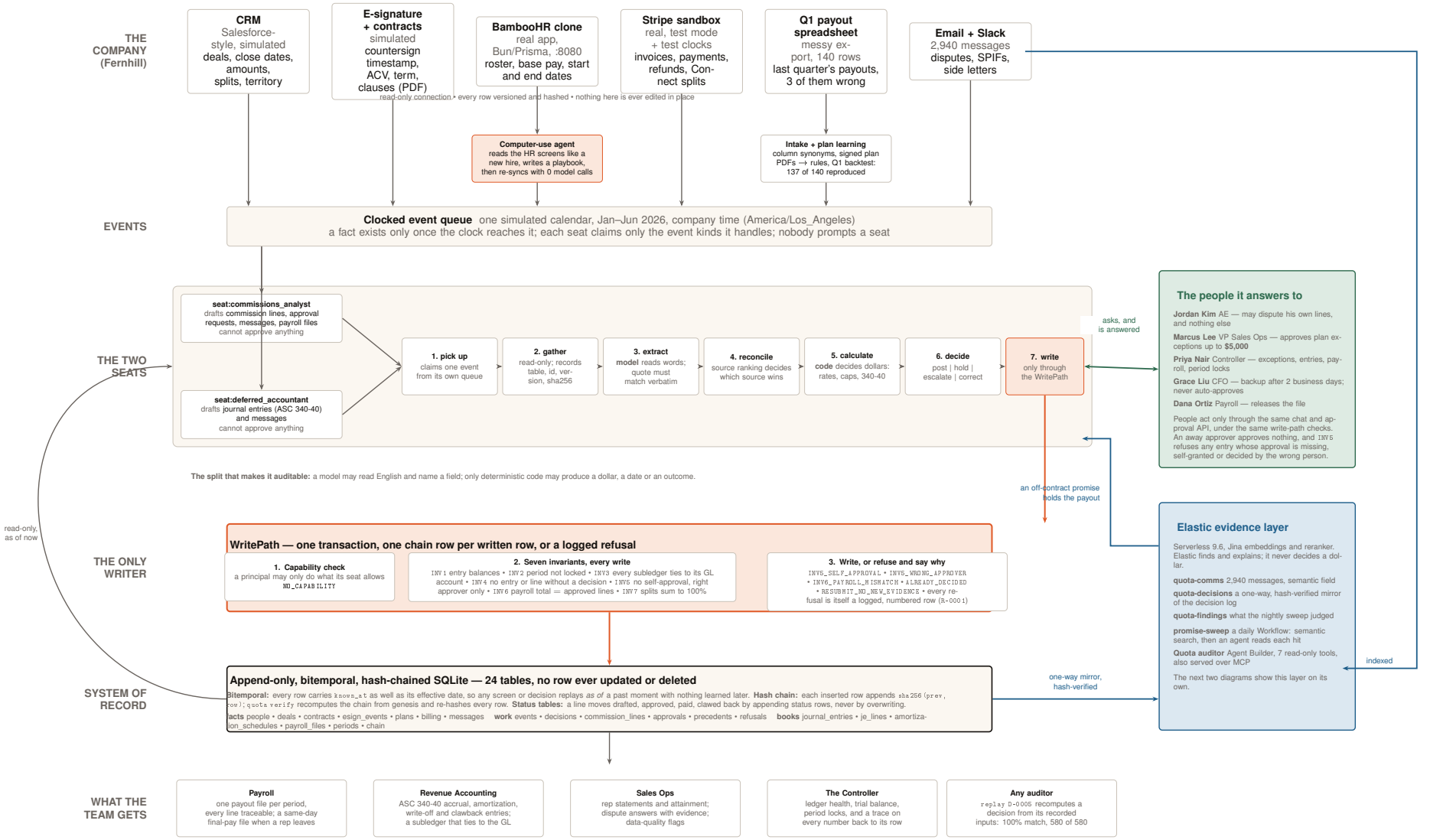
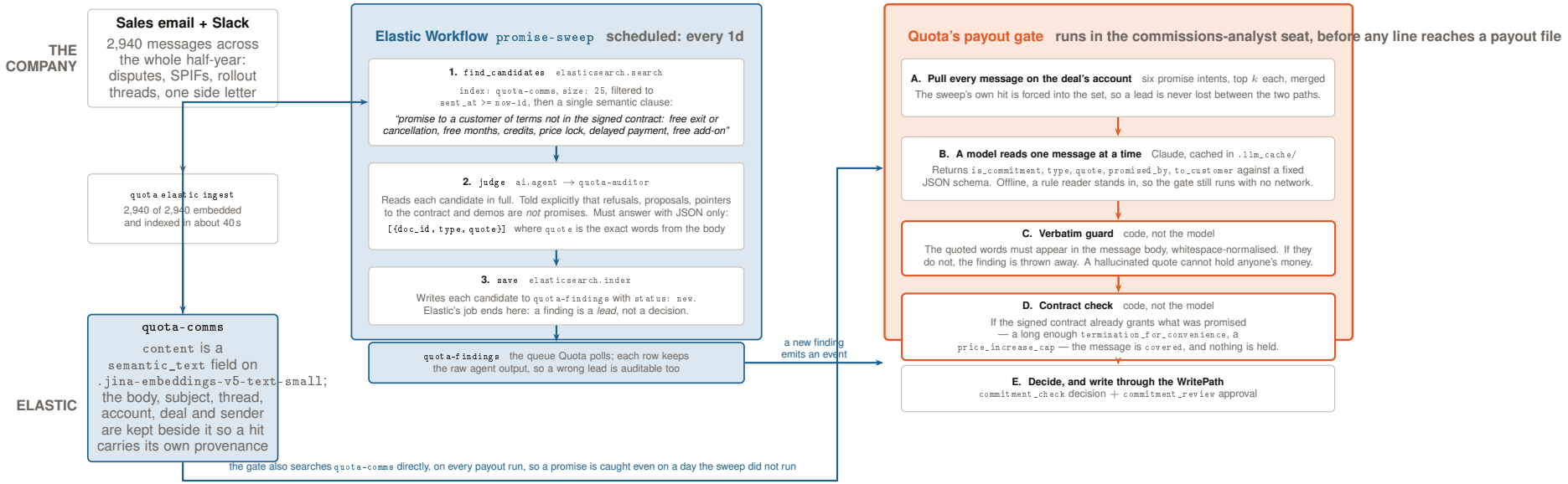


Figure 2 — Elastic, 1 of 2: the promise sweep and the payout gate

How a sentence in an email becomes a $6,000 hold, and the two code checks that stand between a model's reading and someone's paycheck.

Elastic in the workflow, 1 of 2: the off-contract promise sweep and the payout gate

A rep emails a customer a promise the signed contract never made. Nothing in the CRM, the contract or the ledger records it, so no amount of double-entry discipline finds it. Elastic finds it by meaning; deterministic code then decides whether it is real and whether it holds a payout. **Elastic never writes to the ledger and never decides a dollar.**



THE RESULT

What it caught, in the rehearsed world

Riley Shah emailed the Summit Dental contact:

"Between us, if your team hasn't adopted it by end of Q3 we'll let you out of year two, no penalty. Don't worry about the paperwork."

MSG-SIDE-0001, subject "Re: rollout plan", sent 2026-05-05, account Summit Dental, side_letter.jsonl row 1. The signed contract has no early-exit term, so the quote survives both code checks.

D-0014 commitment_check, outcome **held**, **$6,000.00**, reason OFF_CONTRACT_COMMITMENT, chain seq 361.

A-0009 commitment_review opened for Priya Nair, the Controller, for the same $6,000. The line is out of the May payroll file until she answers; the Controller is also told that revenue may be misstated, because an undisclosed exit right changes the term.

No keyword in that email matches "termination", "cancel" or "clause". BM25 does not find it. Semantic search ranks it first.

MEASURED, NEVER EDITED

Retrieval alone: 12 planted promises and 12 decoys in 2,940 messages

mode	recall@20	decoys in top 20	Spanish (P08)	thread (P09)
keyword (BM25)	5 / 12	3	no	no
semantic (Jina)	**10 / 12**	11	**yes**	no
hybrid + rerank	8 / 12	11	**yes**	no

Recall is corpus-wide: the union over six promise intents, top 20 each. Semantic finds twice what keyword finds, including a promise written in Spanish — and pulls in 11 of the 12 decoys. **That is the point:** retrieval is recall, not precision, which is why a model reads every hit and code checks the quote against the body and the contract. The reranker scores the body against the intent sentence and drops P05 and P12, so it finds fewer than semantic alone. Nothing finds P03 (a paraphrase with no exit words) or P09 (the reply "Yes. Consider it done.").

The gate end to end: retrieval + model + the two code checks

promises correctly held	**12 / 12**
decoys wrongly held	**0 / 12**
cases correct, seed 7 and held-out seed 1729	**52 / 52**

The gate is scored offline, where it reads every message on the deal's account rather than the top 20 of a corpus-wide sweep — the same code, given the whole account. The sweep is how a promise is *found* on a day nobody is looking; the gate is what refuses to pay until a person answers.

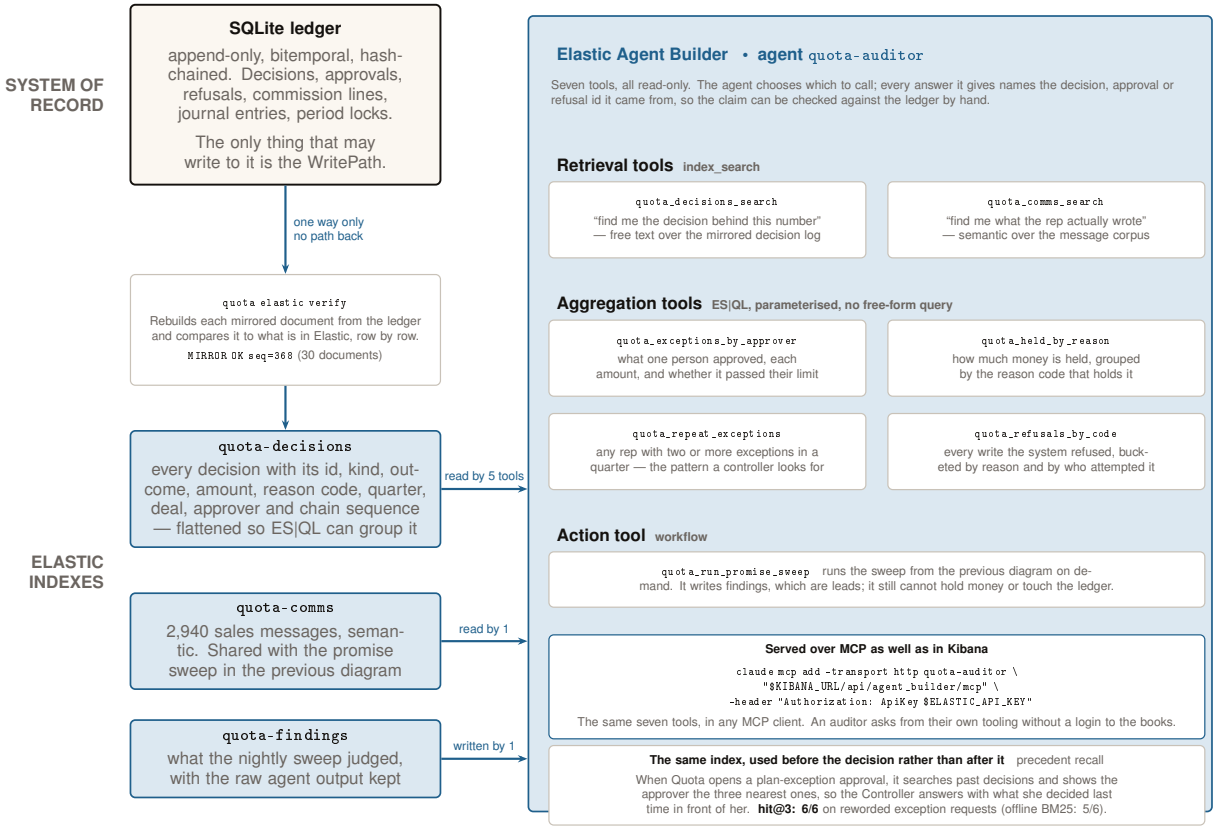
The boundary that makes this safe to run unattended: Elastic proposes, the model reads English, and only deterministic code may hold money. Each layer can be wrong on its own without money moving wrongly.

Figure 3 — Elastic, 2 of 2: the decision mirror and the auditor

A one-way, hash-verified mirror of the decision log, seven read-only tools, and the live rehearsal — including the question it answered wrongly, and why.

Elastic in the workflow, 2 of 2: the decision mirror and the auditor who can be questioned

A ledger that can be questioned in English is worth more at an audit than one that can only be queried in SQL. Quota mirrors its decision log into Elastic **one way** and hash-verified, and puts an Agent Builder agent in front of it with seven read-only tools. The books stay in SQLite; Elastic answers questions about them.



ASKED AND ANSWERED

The rehearsal, run live against the project — including the one it got wrong

Question, asked in English	Tools the agent chose	Time	What came back

"Why is Riley's Summit commission held? Show the evidence."	`held_by_reason`
	`decisions_search`
	`comms_search`
		40.0 s	**Right.** `D-0014`, $6,000.00, `OFF_CONTRACT_COMMITMENT`, the exact quote with its file and row, and `A-0009` still pending with Priya Nair.

"What did Marcus approve this half, and was any of it over his limit?"	`exceptions_by_approver`
	`decisions_search`
	`refusals_by_code`
		102.3 s	**Partly wrong, and the fault was ours.** The limit finding was right (`A-0001`, $6,000 over a $5,000 limit, routed on as `A-0002`), but the mirror had taken Marcus's grant message as the rep, so the agent said he approved "for himself". **Fixed:** the mirror now names a rep only for an AE, and `quota_exceptions_by_approver(marcus.lee)` returns one approval, $6,000, approved, with no rep.

"Did anyone try to bypass a control?"	`refusals_by_code`
	`decisions_search`
	`exceptions_by_approver`
		39.9 s	**Right on the refusal:** `R-0001`, the analyst seat attempting to decide `A-0002`, refused with `NO_CAPABILITY`. One sentence about who had already approved what was wrong, from the same mirror bug.

"Any rep with repeated exceptions in 2026Q2?"	`repeat_exceptions`	—	The agent's model hit Elastic's trial rate limit, so the agent's own ES|QL tool was run directly through `/api/agent_builder/tools/_execute`: **no rows**. No rep has two or more exceptions in the quarter.